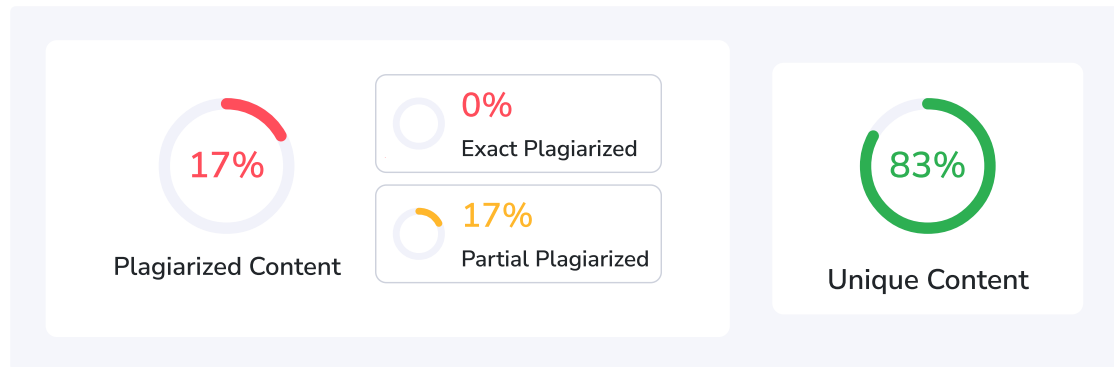


Plagiarism Scan Report By SmallSEOTools

Report Generated on: May 03,2025



Total Words: 860

Total Characters: 7411

Plagiarized Sentences: 5.95

Unique Sentences: 29.05 (83%)

Excluded URL (s)

1 - <https://medium.com/@yusdiansyahandika/mari-kenalan-dengan-socket-bukan-socket-listrik-tapi-websocket-487340b7aa67>

Content Checked for Plagiarism

Sign up Sign in Sign up Sign in Mari kenal dengan Socket! bukan socket listrik tapi WebSocketYansyaFollow3 min read-Just now--ShareDalam era internet yang semakin dinamis, kebutuhan akan aplikasi yang mampu menyajikan data secara real-time dan interaktif semakin meningkat. Aplikasi seperti chatting, game online, dasbor monitoring, dan notifikasi push memerlukan mekanisme komunikasi yang efisien dan responsif. Protokol HTTP, yang mendominasi komunikasi web sejak awal, memiliki keterbatasan dalam menangani skenario real-time murni karena model komunikasinya yang berbasis request-response dan bersifat stateless. Untuk mengatasi keterbatasan ini, protokol WebSocket hadir sebagai solusi yang menyediakan komunikasi duplex penuh dan persisten, membuka jalan bagi pengembangan aplikasi real-time yang lebih baik. Apa itu WebSocket? WebSocket adalah protokol komunikasi yang distandarisasi oleh IETF dalam RFC 6455 (Fette & Melnikov, 2011). Protokol ini memungkinkan komunikasi dua arah (bidirectional) secara simultan antara client dan server melalui sebuah koneksi TCP yang tetap terbuka. Proses koneksi WebSocket dimulai dengan sebuah handshake HTTP biasa. Client mengirimkan permintaan upgrade ke server, yang jika mendukung WebSocket, akan merespons dengan status 101 Switching Protocols, menandakan peralihan ke protokol WebSocket.Dimana WebSocket digunakan?Fleksibilitas dan kemampuan real-time yang ditawarkan WebSocket menjadikannya pilihan ideal untuk berbagai jenis aplikasi:Aplikasi Chat dan Pesan Instan: Memungkinkan pertukaran pesan instan antar pengguna (Singhal, 2013).Game Online Multiplayer: Sinkronisasi data game secara real-time antar pemain.Dasbor dan Visualisasi Data Real-Time: Menampilkan pembaruan data langsung (Perera et al., 2014).Notifikasi Push: Pengiriman pemberitahuan instan dari server ke client.Aplikasi Kolaborasi Real-Time: Memungkinkan kolaborasi simultan pada dokumen atau proyek.IoT (Internet of Things): Komunikasi dua arah antara perangkat dan server.Streaming Data Langsung: Pengiriman stream data berkelanjutan.Aplikasi Trading dan Keuangan: Pembaruan harga dan data pasar secara real-time.Perbedaan antara WebSocket dan HTTPContoh sederhanaBerikut ini adalah program sederhana menggunakan node.js sebagai server dan HTML sebagai client.server.jsconst WebSocket = require('ws');const express = require('express');const path = require('path');const app = express();

```

const port = 8080;

app.get('/', (req, res) => {
  res.sendFile(path.join(__dirname, 'client.html'));
});

const server = app.listen(port, () => {
  console.log(` HTTP server berjalan di http://localhost:${port}`);
});

const wss = new WebSocket.Server({ server });

wss.on('connection', (ws) => {
  console.log('Client tersambung');

  ws.on('message', (message) => {
    console.log('Received:', `${message}`);
    ws.send(` Server menerima: ${message}` );
  });

  ws.on('close', () => {
    console.log('Client disconnected');
  });
});

```

Send

```

const ws = new WebSocket('ws://localhost:8080');
const messagesDiv = document.getElementById('messages');

ws.addEventListener('open', (event) => {
  addMessageToDiv('Connected to server');
});

ws.addEventListener('message', (event) => {
  addMessageToDiv(` ${event.data}`);
});

function sendMessage() {
  const input = document.getElementById('messageInput');
  const message = input.value;
  if (message) {
    ws.send(message);
    addMessageToDiv(` Kamu mengirim: ${message}`);
    input.value = "";
  }
}

function addMessageToDiv(message) {
  const p = document.createElement('p');
  p.textContent = message;

```

```
messagesDiv.appendChild(p);  
}
```

maka jika menjalankan program tersebut di localhost:8080 akan seperti:KesimpulanWebSocket telah menjadi teknologi fundamental untuk pengembangan aplikasi web real-time. Kemampuannya untuk menyediakan komunikasi dupleks penuh dan persisten dengan overhead rendah secara signifikan meningkatkan pengalaman pengguna dalam berbagai aplikasi interaktif. Memahami perbedaan mendasar antara WebSocket dan HTTP, serta mengeksplorasi berbagai use case-nya, memungkinkan pengembang untuk memilih teknologi yang tepat sesuai dengan kebutuhan aplikasi mereka. Dengan terus berkembangnya kebutuhan akan interaktivitas dan real-time di web, peran WebSocket diprediksi akan semakin krusial di masa depan.

Referensi Fette, I., & Melnikov, A. (2011). RFC 6455 — The WebSocket Protocol. IETF. Hossain, M. A., & Islam, M. M. (2019). Performance analysis of WebSocket protocol for real-time web applications. Journal of King Saud University — Computer and Information Sciences, 31(4), 478–486. Perera, S., et al. (2014). Real-time data visualization using WebSockets and server-sent events. International Journal of Computer Science and Information Technology, 6(3), 23–32. Singhal, S. (2013). Real-time web technologies: A comparative study. International Journal of Computer Applications, 68(10). -- Share

Dalam era internet yang semakin dinamis, kebutuhan akan aplikasi yang mampu menyajikan data secara real-time dan interaktif semakin meningkat. Aplikasi seperti chatting, game online, dasbor monitoring, dan notifikasi push memerlukan mekanisme komunikasi yang efisien dan responsif. Protokol HTTP, yang mendominasi komunikasi web sejak awal, memiliki keterbatasan dalam menangani skenario real-time murni karena model komunikasinya yang berbasis request-response dan bersifat stateless. Untuk mengatasi keterbatasan ini, protokol WebSocket hadir sebagai solusi yang menyediakan komunikasi dupleks penuh dan persisten, membuka jalan bagi pengembangan aplikasi real-time yang lebih baik. WebSocket adalah protokol komunikasi yang distandarisasi oleh IETF dalam RFC 6455 (Fette & Melnikov, 2011). Protokol ini memungkinkan komunikasi dua arah (bidirectional) secara simultan antara client dan server melalui sebuah koneksi TCP yang tetap terbuka. Proses koneksi WebSocket dimulai dengan sebuah handshake HTTP biasa. Client mengirimkan permintaan upgrade ke server, yang jika mendukung WebSocket, akan merespons dengan status 101 Switching Protocols, menandakan peralihan ke protokol WebSocket. Fleksibilitas dan kemampuan real-time yang ditawarkan WebSocket menjadikannya pilihan ideal untuk berbagai jenis aplikasi: Berikut ini adalah program sederhana menggunakan node.js sebagai server dan HTML sebagai client. server.js client.html maka jika menjalankan program tersebut di localhost:8080 akan seperti: WebSocket telah menjadi teknologi fundamental untuk pengembangan aplikasi web real-time. Kemampuannya untuk menyediakan komunikasi dupleks penuh dan persisten dengan overhead rendah secara signifikan meningkatkan pengalaman pengguna dalam berbagai aplikasi interaktif. Memahami perbedaan mendasar antara WebSocket dan HTTP, serta mengeksplorasi berbagai use case-nya, memungkinkan pengembang untuk memilih teknologi yang tepat sesuai dengan kebutuhan aplikasi mereka. Dengan terus berkembangnya kebutuhan akan interaktivitas dan real-time di web, peran WebSocket diprediksi akan semakin krusial di masa depan.

Referensi -- -- Help Status About Careers Press Blog Privacy Rules Terms Text to speech